Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la ingeniería

Ingeniería

Organización de Lenguajes y Compiladores 1 Sección “A”

Ing. Jose Moises Granados Guevara

Primer Semestre 2026

Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

Manual Técnico Práctica #2

Descripción:

En el siguiente documento se dan a conocer los detalles sobre cómo fue realizado el proyecto. En este caso es una combinación de varios lenguajes y formas de comunicarse con una aplicación. Se le presenta un proyecto donde gracias al LALR Jison se genera una gramática que permite generar analizadores sintácticos LL(1). Donde gracias a la generación dinámica con el apoyo de una API REST se pueden inyectar dinámicamente nuevos analizadores sintácticos LL(1).

“Bienvenido programador”

Como primer punto se requiere contar con los pasos para poder ejecutar el código del repositorio en la computadora local del programador, al igual que conocer la distribución de carpetas que se hizo. No sin antes mencionar todas las metodologías, tecnologías y patrones de diseño utilizados.

Tecnologías utilizadas para la realización del proyecto:

- **Svelte:** Framework que representa todo el FrontEnd de la aplicación web.
- **NodeJs:** Backend de la aplicación web que permitía comunicarse con los controladores instalados dentro de la app como jison y otras librerías utilizadas.
- **JavaScript:** Principal encargado de toda la lógica de negocio que representa la aplicación y que se requieren para poder validar, ensamblar y utilizar los analizadores sintácticos que esta aplicación permite generar.
- **HTML:** Principal manejo de la estructura de la página web.
- **CSS:** Utilizado para poder darle estilos visuales y agradables a la UI.
    - **Bootstrap:** Principal librería utilizada para hacer una UI intuitiva.
    - **Nativo:** También la app tiene css propio que permite dar una mejor paleta de colores a la aplicación web.
- **Jison: Es un LALR(1).** El principal ayudante de toda la lógica de negocio para crear los analizadores sintácticos que permiten validar las entradas que vienen de la consola de la página web.
- **Java:** Es el encargado de manejar la lógica de negocio Cliente-Servidor que se presenta en la aplicación web. Permitiendo dar un backend **fuertemente tipado. Para poder gestionar transacciones y manejar recursos de persistencia de la web.**
- **SQL:** Es el lenguaje de base de datos utilizado como almacén de persistencia de la aplicación web y respetando el modelo MVC.

Patrones de diseño:

- Patrón experto
- Patrón controlador
- Patrón creador
- Patrón Visitor
- Patrón Singleton

Paradigmas utilizados:

- Programación orientado a objetos
- Programación funcional

Otras tecnologías utilizadas:

- Modelo MVC
- API REST

Debido a que la aplicación fue desarrollada en el entorno NodeJs y utilizando como librería Svelte, teniendo como backend Java. Se recomiendan los siguiente editores de texto o IDE's:

- Netbeans o IntelliJ IDEA
- Tener instalada la versión 22.20.0 como mínimo de **NodeJs**
- **Tener instalado jison como minimo la version: 0.4.18**
- **Visual studio code o cualquier otro editor de texto**
- **Tener instalado minimo la version 17 de jdk**

Pasos iniciales:

1. Encender la computadora.
2. Crear una carpeta donde se va a clonar el repositorio.
3. Ejecutar el comando:

   git clone https://github.com/pablo-mald03/Practica-No2-Compi-1.git
4. Tras descargar el repositorio se puede abrir dentro de cualquier editor de texto que sea de preferencia del programador.
5. Al abrirlo se deben de aceptar los derechos

Pasos de desarrollo:

Se le adjuntan comandos importantes para poder debuggear o poder ir probando la aplicación tanto como frontend como backend.

- **npm install**
- **npm run dev - - - -open**
- **npm run build**
- **npm install jison**
- **npm run parser**

Si en dado caso se le pide descargar una extensión de **Svelte.** Queda a discreción del programador si apoyarse de la extensión. O simplemente guiarse de cómo funciona el framework.

DIRECTORIO DE ARCHIVOS

Debido a que el proyecto base se realizó con cierta distribución de archivos. Así que se le recomienda al nuevo programador que mantenga el estándar que se maneja en el nombramiento de Clases o carpetas. Ya que como tal aquí si es muy importante la distribución de carpetas por **la forma en la que svelte maneja el enrutamiento de la web**. **Tambien es importante destacar que se utilizó el paradigma POO en JavaScript asi que es importante mantener la delegación de tareas para aumentar la Cohesión del proyecto.**

Detalles importantes:

- **La carpeta "analizador":** es donde se encuentra toda la lógica del traductor y de la demás lógica de negocio que maneja la aplicación en el ámbito del LALR que crea Jison.
- **La carpeta "routes":** es donde se manejan los datos que se comparten vía enrutamiento a frontend par que este pueda interpretarlos
- **La carpeta: "services":** es donde se manejan los controladores que se comunican hacia los Endpoints de la API.
- **La carpeta "models":** Maneja los DTO con los que se envía el contenido hacia la API.
- **La carpeta "backend":** Es donde se encuentra toda la lógica de negocio de la aplicación para poder generar todos los comportamientos que esta misma tiene del lado del cliente.

Paquete "ui"

Estas son todas las vistas. Para mantener el formato se recomienda delegar enrutamientos de la APP web a carpetas dentro de **routes.** Ya que aca es donde se cargan las vistas y es muy importante mantener el estandar**:**

+page**.svelte**

Este es un convenio general para evitar revolver código y confundir vistas sobre todo para futuros demás programadores.

IMPORTANTE:

Si se desean crear más paquetes es válido. Solo que si se debe de respetar la distribución base de los paquetes ya mencionados para seguir manteniendo el formato de orden. Sobre todo si hay que agregar documentación se le pide al programador que lo adjunte dentro de la ruta llamada **documentacion.**

ANÁLISIS DE GRAMÁTICA PARA ANALIZADOR LÉXICO

En el presente proyecto se lleva a cabo un análisis léxico gracias a Jison que permite generar un **autómata finito determinista** en base a los principios del método de Thompson hasta poder llegar a un autómata con todos los estados que permita validar los estados de aceptación. Por lo tanto el método consiste en:

- Declaración de Expresiones regulares
- Generación de **AFN por el método de Thompson**
- Formulación de **AFD en base al algoritmo de cerradura Lambda**
- **Minimización de estados**

Aca se adjuntan todas las expresiones regulares utilizadas en el lexer:

```
/* Sección analizador Léxico */

%lex

%options ranges yylineno

%%

/*Espacios y saltos de línea*/


\s+                    /* ignorar espacios */


[\u200B\uFEFF\u200E\u200F\u202A-\u202E]+ /* Ignorar caracteres basura */


/*Apartado de comentarios*/


"#".*            /*Ignorar comentario de linea*/


"/**"([^*]|\*+[^*/])*(\*+"/")           /*Ignorar comentario multilinea*/
```

```
/*---*******------Expresiones regulares reglas lexicas aceptadas en el
lenguaje-----********---*/

"[0-9]"                    return 'RANGO_NUMERO';

"[a-z]"                    return 'RANGO_MINUSCULA';

"[A-Z]"                    return 'RANGO_MAYUSCULA';

"[aA-zZ]"                    return 'RANGO_TOTAL';


/*---*******------Palabras reservadas del lenguaje-----********---*/

"Wison"                        return 'WISON';

"Lex"                          return 'LEX';

"Terminal"                     return 'TERMINAL';

"No_Terminal"                  return 'NO_TERMINAL';

"Syntax"                       return 'SYNTAX_PARSER';

"Initial_Sim"                  return 'SIMBOLO_INICIAL';
```

```
/*---*******------Caracteres especiales del lenguaje------********---*/

"("                         return 'PARENT_APERTURA';

")"                         return 'PARENT_CIERRE';

"{"                         return 'LLAVE_APERTURA';

"}"                         return 'LLAVE_CIERRE';

"["                         return 'CORCHETE_APERTURA';

"]"                         return 'CORCHETE_CIERRE';

":"                         return 'DOS_PUNTOS';

"<"                         return 'MENOR';

";"                         return 'PUNTO_COMA';

"|"                         return 'OR_EXP';
```

```
/*---*******------Caracteres que permiten usar expresiones regulares en el
lenguaje wison-----*******---*/

"-"                         return 'MENOS';

"+"                         return 'MAS';

"*"                         return 'KLEENE';

"¿"                         return 'QUEST_APERTURA';

"?"                         return 'QUEST_CIERRE';

"$_"[a-zA-Z0-9_]+           return 'ID_TERMINAL';

"%_"[a-zA-Z0-9_]+           return 'ID_NO_TERMINAL';

"'"[^']*"'"             {
                        yytext = yytext.slice(1, -1);
                        return 'LITERAL_CADENA';
                        }



/*---*******------Caracteres que permiten generar producciones en el
analizador sintactico-----*******---*/
```

```
"="                         return 'IGUAL';


.                           {
                                if (!yy.errores) yy.errores = [];


                                yy.errores.push({
                                    lexema: yytext,
                                    tipo: "Lexico",
                                    fila: yylloc.first_line,
                                    columna: yylloc.first_column + 1,
                                      descripcion: "Caracter no valido en el
lenguaje"
                                });
                            }


<<EOF>>                     return 'EOF';


/lex
```

ANÁLISIS DE GRAMÁTICA PARA ANALIZADOR SINTÁCTICO LALR(1)

Cabe aclarar que aquí simplemente se están agregando los **terminales** y los **no terminales** debido a que la gramática es muy extensa. pero simplemente se puede visualizar la cantidad de derivaciones que esta tiene. Puesto a que los errores se debían detectar desde el Parser el código es bien extenso. Por lo tanto par más información consultar el archivo llamado

wison-config.jison

```
/*----Simbolo inicial----*/


%start inicio


%%


/*-----==========-----Produccion principal de inicio-----=====-----*/


inicio  : WISON QUEST_APERTURA cuerpo  QUEST_CIERRE WISON  EOF

        {

        }

        | error EOF

        {

        }

        ;
```

```
/*----==========-----Produccion del cuerpo general de Wison----=====-----*/


cuerpo      : bloque_lexico bloque_sintactico

            {{

            }}

            ;




/*-----==========-----Produccion para el bloque lexico----=====-----*/


bloque_lexico   : LEX LLAVE_APERTURA DOS_PUNTOS estructura_lexica DOS_PUNTOS
LLAVE_CIERRE

                {{

                }}

                | error LLAVE_CIERRE

                {{

                }}

                ;
```

```
/*----==========-----Produccion para el bloque sintactico----=====-----*/


bloque_sintactico   : SYNTAX_PARSER LLAVE_APERTURA LLAVE_APERTURA DOS_PUNTOS
estructura_sintactica DOS_PUNTOS LLAVE_CIERRE LLAVE_CIERRE

                    {{

                    }}

                    | error LLAVE_CIERRE LLAVE_CIERRE

                    {{

                    }}

                    ;


/*----==========-----Produccion     que     define     a     la     estructura
lexica----=====-----*/


estructura_lexica

                : estructura_lexica produccion_terminal

                {{

                }}

                | produccion_terminal

                {{

                }}

                ;
```

```
/*----==========-----Producciones        individuales        del        apartado
lexico----=====-----*/


produccion_terminal  :  TERMINAL  ID_TERMINAL    MENOR  MENOS  regla_lexica
PUNTO_COMA

                    {{

                    }}

                    | TERMINAL ID_TERMINAL  MENOR MENOS  regla_lexica

                    {{

                    }}

                    | TERMINAL ID_TERMINAL  MENOR MENOS  PUNTO_COMA

                    {{

                    }}

                    | TERMINAL ID_TERMINAL

                    {{

                    }}

                    | TERMINAL

                    {{

                    }}

                    | error PUNTO_COMA

                    {{

                    }}

                    ;
```

```
/*----==========-----Producciones        individuales        del        apartado
lexico-----=====-----*/


regla_lexica    : regla_lexica elemento_lexico

                {{

                }}

                | elemento_lexico

                {{

                }}

                ;




/*----==========-----Producciones  para  elementos  lexicos  principales  definidos
en la seccion del lexer----==========-----*/

elemento_lexico : unidad_lexica modificador

                {{

                }}

                | unidad_lexica

                {{

                }}

                ;
```

```
/*----==========-----Producciones para las reglas de operadores unarios
permitidos-----==========-----*/


modificador : KLEENE

            {

            }

            | MAS

            {

            }

            | QUEST_CIERRE

            {

            }

            ;



/*----==========-----Producciones para las expresiones regulares
permitidas-----==========-----*/


unidad_lexica   : LITERAL_CADENA

                {{

                }}

                | PARENT_APERTURA regla_lexica PARENT_CIERRE

                {{

                }}
```

```
                | ID_TERMINAL
                {{
                }}
                | RANGO_NUMERO
                {{
                }}
                | RANGO_MINUSCULA
                {{
                }}
                | RANGO_MAYUSCULA
                {{
                }}
                | RANGO_TOTAL
                {{
                }}
                ;

/*----==========-----Producciones   para   las   transiciones   del   Analizador
Sintactico-----==========-----*/
estructura_sintactica       : lista_instrucciones_sintacticas
                            {
                            }
                            ;
```

```
/*----==========-----Producciones para la lista de transiciones del analizador
sintactico----==========-----*/


lista_instrucciones_sintacticas          :  lista_instrucciones_sintacticas
instruccion_sintactica
                                    {{
                                    }}
                                    | instruccion_sintactica
                                    {{
                                    }}
                                    ;


/*----==========-----Producciones que definen a las instrucciones sintacticas
que hay----=====-----*/


instruccion_sintactica : non_terminales
                    {{
                    }}
                    | sim_inicial
                    {{
                    }}
                    | regla_produccion
                    {{
                    }}
                    | error PUNTO_COMA
                    {{
```

```
                        });

                    }}

                    ;




/*----==========-----Produccion     de     la     estructura     de     los     no
terminales-----==========-----*/


non_terminales      : NO_TERMINAL ID_NO_TERMINAL PUNTO_COMA

                    {{

                    }}

                    ;




/*-----==========-----Produccion     del     simbolo     inicial     de     la
gramatica-----==========-----*/


sim_inicial         : SIMBOLO_INICIAL ID_NO_TERMINAL PUNTO_COMA

                    {{

                    }}

                    ;
```

```
/*----==========-----Produccion que define a la declaracion de las reglas de
producion de la gramatica-----=====-----*/


regla_produccion        :  ID_NO_TERMINAL  MENOR  IGUAL  lista_alternativas
PUNTO_COMA

                    {{

                    }}

                    ;




/*----==========-----Produccion que define a la declaracion de las alternativas
de la produccion o del no terminal-----=====-----*/


lista_alternativas  : lista_alternativas OR_EXP cuerpo_produccion

                    {{

                    }}

                    | cuerpo_produccion

                    {{

                    }}

                    ;
```

```
/*-----==========-----Produccion que define al cuerpo de las producciones de
Wison-----=====-----*/


cuerpo_produccion : lista_simbolos

                    {{

                    }}

                    | /*Produccion vacia*/

                    {{

                    }}

                    ;




/*-----=====-----Produccion que define al listado de producciones dentro  de
Wison-----=====-----*/

lista_simbolos      : lista_simbolos simbolo

                    {{

                    }}

                    | simbolo

                    {{

                    }}

                    ;
```

```
/*----------=========-----Produccion que a todas las variantes que tienen las
producciones de la gramatica----=====-----*/


simbolo     : ID_TERMINAL

            {{

            }}

            | ID_NO_TERMINAL

            {{

            }}

            ;
```

IMPORTANTE:

Debido a que los operadores unarios para este lenguaje no requieren precedencia debido a que son textos que siguiendo una misma producción se pueden validar. Sin embargo si existe la función precedencia para poder definir una gramática más fuerte. Si en algún futuro se desea ampliar más funcionalidades tomarlo en cuenta.

GENERACIÓN DE LAS GRAMÁTICAS

Para este proyecto en específico es importante enfatizar en todo el algoritmo y pasos para poder generar las gramáticas. Ya que como tal no se tiene un catálogo de gramáticas. Sino que simplemente es una aplicación de todos los temas que se aprenden en el curso de compiladores. Es una implementación algorítmica de todo el proceso que hay que seguir para poder llegar a crear una **tabla LL(1).** Y con esta tabla se puede generar algorítmicamente código que permite realizar transiciones. Media vez se tiene la tabla de análisis sintáctico se puede llegar a un algoritmo recursivo.

A continuación se le dará a conocer un pseudocódigo de cómo se implementa la lógica para poder ejecutar los distintos algoritmos que se plantean tanto en teoría como se pasa por las diferentes pasadas en donde se hacen los cálculos para poder armar la tabla de análisis sintáctico y llegar al algoritmo recursivo.

Cálculo de primeros:

```
calcularPrimeros() INICIO
        var huboCambios = true


        MIENTRAS (huboCambios == true) HACER
            huboCambios = false
            PARA CADA instruccion DE bloqueSintactico INICIO
                SI (instruccion.tipo != 'PRODUCCION') ENTONCES
                    CONTINUAR
                FINSI
                noTerminal = instruccion.padre
                setActual = primeros.get(noTerminal)
                longitudAntes = setActual.size
```

```
PARA CADA alternativa DE instruccion.alternativas INICIO

                todosPuedenSerLambda = true

                PARA CADA simbolo DE alternativa INICIO
                    SI (simbolo.tipo == 'TERMINAL' || simbolo.tipo ==
'LAMBDA') ENTONCES
                        setActual.add(simbolo.valor)
                        todosPuedenSerLambda = false
                        ROMPER
                    SINO (simbolo.tipo == 'NO_TERMINAL') ENTONCES
                                                const primerosHijo =
this.primeros.get(simbolo.valor) || new Set();
                        hijoTieneLambda = false

                        PARA CADA token DE primerosHijo INICIO
                            SI (token == 'LAMBDA') ENTONCES
                                hijoTieneLambda = true
                            SINO
                                setActual.add(token)
                            FINSI
                        FINPARA

                        SI (hijoTieneLambda == false) ENTONCES
                            todosPuedenSerLambda = false
                            ROMPER
```

```
                        FINSI


                    FINPARA


                FINSI


                SI (todosPuedenSerLambda) ENTONCES
                    setActual.add('LAMBDA')
                FINSI


            FINPARA


            SI (setActual.size > longitudAntes) ENTONCES
                huboCambios = true;
            FINSI
        FINPARA
    FINMIENTRAS
FIN
```

Cálculo de siguientes:

```
calcularSiguientes() INICIO

        bloqueSintactico = ast.sintactico

        idSimboloInicial = null


        PARA CADA instruccion DE bloqueSintactico INICIO


            SI (instruccion.tipo == 'SIMBOLO_INICIAL') ENTONCES

                idSimboloInicial = instruccion.id

                ROMPER

            FINSI

        FINPARA


        SI (idSimboloInicial && siguientes.has(idSimboloInicial))
ENTONCES

            siguientes.get(idSimboloInicial).add('EOF')

        FINSI




        huboCambios = true


        MIENTRAS (huboCambios == true) HACER


            huboCambios = false


            PARA CADA instruccion DE bloqueSintactico HACER
```

```
                IF (instruccion.tipo != 'PRODUCCION') ENTONCES
                    CONTINUAR
                FINSI

                padre = instruccion.padre

                PARA CADA alternativa DE instruccion.alternativas
ENTONCES



                    PARA i = 0 MIENTRAS i < alternativa.length CON PASO
i++ HACER

                        simboloActual = alternativa[i]

                        SI (simboloActual.tipo != 'NO_TERMINAL') ENTONCES
                            CONTINUAR
                        FINSI

                        setSiguienteHijo =
siguientes.get(simboloActual.valor)
                        SI (setSiguienteHijo == false) ENTONCES
                            CONTINUE
                        FINSI

                        longitudAntes = setSiguienteHijo.size
                        todoLoQueSigueEsLambda = true
```

```
                        PARA  j = i + 1 HACER j < alternativa.length CON
PASO j++ HACER
                            simboloAdelante = alternativa[j
                            SI (simboloAdelante.tipo == 'TERMINAL')
ENTONCES

setSiguienteHijo.add(simboloAdelante.valor)
                                todoLoQueSigueEsLambda = false
                                ROMPER
                            FINSI
                            SINO (simboloAdelante.tipo === 'NO_TERMINAL')
{

                                primerosAdelante =
this.primeros.get(simboloAdelante.valor) || new Set()
                                adelanteTieneLambda = false

                                PARA CADA token DE primerosAdelante)
HACER
                                    SI (token === 'LAMBDA') ENTONCES
                                        adelanteTieneLambda = true
                                    SINO
                                        setSiguienteHijo.add(token)
                                    FINSI
                                FINPARA
                                SI (adelanteTieneLambda == false) FINSI
                                    todoLoQueSigueEsLambda = false
                                    ROMPER
                                FINSI
```

```
                        FINSI


                    FINPARA


                    SI (todoLoQueSigueEsLambda) ENTONCES
                        const siguientesDelPadre =
this.siguientes.get(padre) || new Set();
                        PARA CADA token DE siguientesDelPadre) HACER
                            setSiguienteHijo.add(token);
                        FINPARA
                    FINSI


                    SI (setSiguienteHijo.size > longitudAntes)
ENTONCES
                        huboCambios = true;
                    FINSI
                FINPARA
            FINPARA
        FINPARA
    FINMIENTRAS
FIN
```

Después de calcular los primeros y siguientes se procede a agregar a la tabla LL(1). Se procede a generar el código que se va a procesar. Se recorre por cada no terminal y las transiciones que este genera con los tokens definidos por el usuario. La lógica que se sigue es la siguiente:

- **No terminales:** Se traducen a funciones.
- **Terminales:** Son tokens que se reconocen como enums y esto representa a las transiciones que se hacen de forma recursiva. teniendo como resultado el final del stack.

La forma de generar el parser es completamente recursivo y de esta forma permite generar el árbol. Al ser código JavaScript es super flexible de generar este tipo de código ya que se puede inyectar dentro del DOM de la aplicación web lo que permite generar este dinamismo de la aplicación que es generar **analizadores descendentes**.

DIAGRAMA DE CLASES

Este es uno de los diagramas que describe la jerarquía principal que sigue el intérprete para poder generar las gramáticas, hacer análisis de generación de Tabla LL(1) y la forma de ensamblar las piezas para poderlas convertir en archivos. Simplemente es una jerarquía con métodos básicos. **Cabe destacar que principalmente se pensó para poderse ejecutar en algunas acciones. Pero de momento el proyecto no lo requería por lo tanto fueron eliminadas esas funcionalidades**.

Puedes verlo más a detalle en el link:

https://app.diagrams.net/#G1ZR8dCN_RJfH-DatPq1S-9yKBRsrymgS4#%7B%22pageId%22%3A%229FyhZFpsurD1-RChVbB9%22%7D